
Tamil Sign Language Fingerspelling Recognition

A Comparative Analysis of CNN and Transformer Architectures Across 247 Character Classes

63046¹ 61904¹ 64552¹ 60133¹

Abstract

We present, to our knowledge, the first published deep-learning benchmark on the full 247-class TLFS23 task, comparing EfficientNet-B3 and Swin-Small under matched training regimes across all 247 Tamil characters. Swin-Small achieves 99.35% Top-1 accuracy (Macro-F1 0.9935) against EfficientNet-B3’s 51.77% (Macro-F1 0.4666), a statistically significant gap (McNemar’s $\chi^2(1) = 3497.27$, $p < 0.001$). Beyond this headline difference, the architectures dissociate in error structure: Swin-Small outperforms EfficientNet-B3 on 245 of 247 classes (mean per-class F1 advantage 0.527) and resolves 19 of EfficientNet-B3’s 20 most-confused class pairs. These are pairs that cluster on numerically adjacent indices sharing a base consonant in the Uyir-Mei compositional system. Grad-CAM shows that EfficientNet-B3’s failures coincide with attention drifting onto task-irrelevant background, giving a mechanistic account of where the convolutional locality prior breaks down when discriminative information is compositional rather than local. For compositionally structured Indic sign language tasks, per-class F1 dispersion and confusion-pair topology are more informative than aggregate accuracy.

1. Introduction

Sign language recognition has matured alongside deep learning, but the empirical focus has stayed narrow. Reviews find that American Sign Language dominates published work, that convolutional architectures dominate benchmarking, and that linguistically diverse high-class-count tasks are underrepresented (Khan et al., 2024; Rastgoo et al., 2021; Patel et al., 2024). Tamil is one of the starker examples. Despite a reader community in the tens of millions, the published Tamil SLR literature relies almost entirely on handcrafted

features, MediaPipe keypoints, or classical classifiers on small vocabularies (Prabhu et al., 2024; Karthikeyan et al., 2022; Jayasinghe et al., 2011; Rajkumar et al., 2019). As far as we can tell, no prior work benchmarks modern deep architectures on the full Tamil character set.

The TLFS23 dataset (Singaravel et al., 2023) contains about 255,000 images across 248 classes spanning the vowels, consonants, and *Uyir-Mei* (vowel-consonant compound) characters of written Tamil which makes a serious benchmark feasible. TLFS23 differs from most SLR datasets in two ways. It has a large class count, roughly an order of magnitude beyond typical ASL alphabet datasets; and, more consequentially, the label space is *compositional*. Written Tamil is built from 12 vowels, 18 consonants, and 216 Uyir-Mei compounds formed by pairing a consonant base with a vowel modifier. Visually, each compound shares its base stroke with the parent consonant and differs only in a small positional modifier, precisely the regime where architectural inductive biases are expected to matter most. A model that learns consonant-level identity without modifier-level discrimination achieves roughly $1/12 \approx 8\%$ within-family accuracy by chance, close to the pattern we document for EfficientNet-B3.

We use TLFS23 to ask a precise question: do CNN and transformer architectures produce qualitatively different error structures on a compositionally structured 247-class label space, and can those differences be mechanistically attributed to their inductive biases? CNNs have local receptive fields and translation-equivariant weight sharing, which bias them toward texture and local-pattern discrimination (Tan and Le, 2019). Hierarchical transformers like Swin model long-range dependencies through windowed self-attention with shift-based cross-window propagation (Liu et al., 2021; Dosovitskiy et al., 2020). Whether one is globally stronger is one question; whether the two fail differently is another, and potentially the more useful one.

Our contributions are three. First, to our knowledge the first published deep-learning benchmark on the full 247-class TLFS23 task, with both architectures fine-tuned end-to-end under matched training regimes: identical 70/15/15 splits, identical class indexing, identical class weighting, and a

¹Department of Statistics, London School of Economics and Political Science, London, UK.

40-epoch Stage 2 with cosine annealing for both. Second, a set of error-structure diagnostics: per-class F1 dispersion, confusion-pair topology indexed against TLFS23’s compositional class ordering, Top-1/Top-5 gap, and Macro-vs-Weighted F1 alignment which is supported by Grad-CAM evidence that the CNN’s failures correspond to attention drift onto task-irrelevant background rather than within-hand discrimination failure. Third, an empirically grounded complexity partition of the 247-class label space derived from inter-class cosine similarity in pixel space, used as an architecture-agnostic difficulty axis to test whether per-tier means capture the dissociation; we find they do not, and that the more informative diagnostic is confusion-pair topology indexed against the Uyir-Mei compositional system.

While the 47.6-point Top-1 gap is striking, the more informative finding lies in *how* EfficientNet-B3 fails: nearly identical Macro and Weighted F1 (0.4666 vs. 0.4667), per-class F1 spread of $\sigma \approx 0.24$, a 35-point Top-1/Top-5 gap, and confusion pairs clustered on adjacent class indices. Each is a fingerprint of a model that has learned coarse hand-shape categories but cannot discriminate within compositional sub-structure, the failure mode predicted by a locality prior when the discriminating signal is global and compositional.

2. Related Work

Early SLR systems used classical vision: hand segmentation, pose estimation, and geometric features fed into conventional classifiers (Isaacs and Foo, 2004; Jayasinghe et al., 2011). These pipelines worked under controlled conditions but scaled poorly and were fragile to lighting and background variation. The transition to deep learning (Rastgoo et al., 2021; Khan et al., 2024) established CNNs as the dominant paradigm for static gesture classification; sequence models took over for continuous signing (Duarte et al., 2021). Reviews consistently identify a language coverage gap: research concentrates on ASL, with less on ISL (Mangamuri et al., 2019; Sharma et al., 2024; Singh et al., 2023) and even less on Tamil (Prabhu et al., 2024; Karthikeyan et al., 2022; Rajkumar et al., 2019). No prior published Tamil SLR work we are aware of benchmarks transfer-learned deep architectures on the full character set. TLFS23 (Singaravel et al., 2023) provides the dataset to close this gap; the dataset paper itself offers no benchmark models, and the few subsequent works using TLFS23 evaluate only narrow subsets, which we discuss in detail in Section 5.5.

Existing Tamil SLR work uses MediaPipe landmarks, small CNNs, or HOG+SVM classifiers on vowel-only subsets (Prabhu et al., 2024; Karthikeyan et al., 2022; Rajkumar et al., 2019) we found that none evaluates the 216 Uyir-Mei compounds, which is why the compositional failure mode we identify is not visible in prior work: the classes that

expose it were not being tested.

Our question sits inside the broader problem of fine-grained visual classification, where classes differ only in subtle local features. What distinguishes TLFS23 is not difficulty per se but the *structure* of the discriminating signal. Fine-grained bird classification asks a model to attend to the right local region: a beak shape, a wing bar, without integrating it with another. Compositional fingerspelling asks a model to integrate a base stroke with a modifier in a specific spatial relationship. Locality is sufficient for the first but not the second, and that difference is what our diagnostics isolate.

Finally, for interpretability we rely on Grad-CAM (Selvaraju et al., 2017), which produces class-discriminative localisation maps by weighting convolutional feature maps with the gradients of the class score flowing into the last convolutional layer. Grad-CAM transfers cleanly to EfficientNet-B3, which lets us diagnose the CNN’s failures directly, but does not transfer cleanly to shifted-window self-attention; we flag this asymmetry as a limitation in Section 5.10.

3. Architecture

We compare two architectures chosen to isolate the role of self-attention. Both are pretrained on ImageNet and fine-tuned end-to-end on TLFS23.

EfficientNet-B3 (convolutional baseline). EfficientNet-B3 is an 11.08M-parameter convolutional network derived from EfficientNet-B0 by compound scaling of depth, width, and input resolution (Tan and Le, 2019). It instantiates the CNN inductive bias cleanly: depthwise separable convolutions with translation equivariance and local receptive fields build a feature hierarchy from low-level texture upward.

The effective receptive field of a convolutional stack grows as $r_L = 1 + \sum_{l=1}^L (k_l - 1) \cdot \prod_{j=1}^{l-1} s_j$, where k_l is the kernel size at layer l and s_j is the stride at layer j . For EfficientNet-B3’s depthwise separable layers with $k = 3$ and unit stride, this simplifies to $r_L = 1 + 2L$, reaching approximately 47×47 pixels at the final block on a 300×300 input. The discriminating modifier in a Uyir-Mei compound must be integrated with the base stroke across the full hand span a relational operation that linear receptive field growth cannot guarantee, and the mechanistic account consistent with the locality failure we document in Section 5.8.

The compound scaling rule takes EfficientNet-B0 and scales depth, width, and resolution jointly by a fixed exponential factor, so B3 is a specific point on a one-parameter family of models. We pick B3 as the best fit for our compute budget. We use the `tf_efficientnet_b3_ns_jft_in1k` weights from `timm`, which combine noisy-student pretraining with an ImageNet fine-tune. On top of the backbone we attach a two-layer classification head: dropout ($p = 0.3$)

followed by a linear projection to 247 logits. Inputs are resized to 300×300 and normalised to $[-1, 1]$ using the preprocessing function paired with the pretrained weights.

Swin-Small (transformer arm). Swin-Small is a 48.84M-parameter hierarchical vision transformer (Liu et al., 2021). It first partitions each input into 4×4 patches and projects them to 96-dimensional tokens. Self-attention is then computed within 7×7 non-overlapping windows, which bounds the attention cost at $O(N)$ in the number of tokens rather than $O(N^2)$. The key innovation is the shifted-window scheme: between successive transformer blocks the window partition shifts by half a window, so tokens that shared a window in one layer no longer do in the next, allowing information to propagate across the original window boundaries. The shifted-window scheme ensures that any two tokens separated by at most $2M$ positions share a window in either the regular or the shifted partition; after d consecutive W-MSA/SW-MSA block pairs, information propagates across $2dM$ tokens. Full receptive coverage of the 56×56 Stage 1 feature map is achieved within $d = 4$ block pairs, guaranteeing that base stroke and vowel modifier are jointly available to the classifier regardless of their spatial separation - the property that the CNN’s linear receptive field growth cannot provide.

The feature map is downsampled between stages via patch merging, producing a pyramid of [96, 192, 384, 768] channels at [56, 28, 14, 7] spatial resolutions. That hierarchical structure mirrors the multi-scale spatial hierarchy CNNs produce, which makes Swin a more CNN-comparable transformer than the original ViT. We pick Swin-Small (48.84M) over Tiny (28M, would underfit) and Base (88M, over-capacity); it sits in the same order of magnitude as EfficientNet-B3 while remaining a representative Swin instantiation. We load `microsoft/swin-small-patch4-window7-224` from HuggingFace and attach a head of dropout ($p = 0.3$) plus a linear projection to 247 logits. Inputs are resized to 224×224 (the ImageNet pretraining resolution) and normalised to $[0, 1]$.

The two architectures differ in three ways relevant to interpretation: Swin’s shifted-window propagation gives it cross-window reach at every stage that the CNN’s local receptive fields lack; Swin has $4.4\times$ more parameters (48.84M vs. 11.08M); and input resolutions follow each backbone’s pretraining convention (300×300 vs. 224×224), which we do not equalise to avoid misaligning pretrained feature statistics.

4. Training Methods

Both models follow a two-stage transfer-learning protocol. Stage 1 freezes the pretrained backbone and trains only the

classification head; Stage 2 unfreezes the backbone for end-to-end fine-tuning at a reduced learning rate. The rationale is standard: Stage 1 adapts the head to the new label space without disturbing pretrained features, and Stage 2 adapts the features themselves once the head is no longer random, which is what initially backpropagating through a random head would otherwise damage.

EfficientNet-B3 schedule: Stage 1 runs for up to 10 epochs at Adam learning rate 10^{-3} , training the head only with patience 5 on validation loss. Stage 2 runs for up to 40 epochs at Adam 10^{-5} under cosine annealing, with the entire backbone unfrozen and the same patience policy. The loss is class-weighted cross-entropy with label smoothing 0.1. Class weights are computed as balanced inverse frequency over the training set and range from 0.815 to 1.001 (mean 1.000); at this level of imbalance (see Section 5.2) the marginal effect on optimisation is small, but we apply weighting in both pipelines for symmetry. Training uses a single NVIDIA A100 with batch size 32.

Swin-Small schedule: Stage 1 runs for up to 10 epochs at 10^{-3} (head only); Stage 2 runs for up to 40 epochs at 10^{-5} under cosine annealing with patience 5. The Swin pipeline uses a hybrid TensorFlow/PyTorch implementation in Stage 1 and pure PyTorch in Stage 2. Because Stage 1 freezes the backbone, its outputs on a given image are deterministic, so we extract all backbone features in a single pre-pass and train the Keras head on cached features, which cuts Stage 1 wall-time by roughly an order of magnitude without changing the resulting weights. Stage 2 rebuilds the pipeline end-to-end in PyTorch so gradients flow through all 48.84M parameters. The loss is class-weighted cross-entropy (no label smoothing).

Augmentation: Augmentation is applied only to training images; validation and test images are evaluated on clean inputs. The CNN pipeline uses tiered augmentation scaled to per-class complexity: rotation $\pm 10^\circ$ for simple classes up to $\pm 15^\circ$ for complex, with brightness, contrast, translation and zoom jitter increasing correspondingly. The Swin pipeline uses uniform moderate augmentation (rotation $\pm 10\%$, zoom $\pm 10\%$, brightness $\pm 10\%$) across all classes. Horizontal flip is excluded in both pipelines because Tamil characters are directionally meaningful and a flip would produce a different (or invalid) glyph.

Regularisation and overfitting mitigation: Four mechanisms guard against overfitting. Dropout ($p = 0.3$) on the classification head of both models. Label smoothing 0.1 in the CNN loss (not matched in Swin, which we return to in the Discussion). Early stopping with patience 5 on validation loss in both stages of both pipelines. And the augmentation regimes above, which make the effective training

set larger than its 34,612 raw images.

What the comparison controls for. Held constant: dataset, train/val/test partition, class-index mapping, class-weighting strategy, two-stage frozen-then-unfrozen protocol, optimiser (Adam), Stage 2 epoch budget (40), cosine annealing schedule, and early-stopping policy. Architecture-mandated differences: input resolution and normalisation, both tied to pretraining. Residual pipeline differences: the CNN uses tiered complexity-aware augmentation and label smoothing 0.1; Swin uses uniform moderate augmentation without label smoothing.

Training dynamics: The two runs behave differently in ways that matter for the results that follow, so we document them briefly. EfficientNet-B3 Stage 1 reached validation Top-1 ≈ 0.097 over 10 epochs (Appendix C, Figure 8); Stage 2 hit early stopping at epoch 12 with validation Top-1 ≈ 0.42 and Top-5 ≈ 0.80 - essentially the final test performance. Swin-Small Stage 1 plateaued near 0.01 (Appendix D, Figure 11), confirming frozen ImageNet features are not linearly separable on this 247-class task; Stage 2 reached validation Top-1 ≈ 0.997 by epoch 15 (Figure 12). Training accuracy during Swin Stage 2 sits near 50% due to on-the-fly augmentation applied to training batches; this does not indicate train/val inversion. The close val/test agreement (99.73% vs. 99.29%) rules out overfitting to the validation set.

5. Numerical Results

5.1. Evaluation goals

Our numerical evaluation has two goals that align with two different levels of resolution. The first is straightforward: quantify how well each architecture classifies Tamil fingerspelling under a matched training regime. For this, headline Top-1/Top-5 accuracy and Macro/Weighted F1 on a held-out test set are the standard metrics, and we report them as the opening of our analysis. The second goal is the one this paper is really about: characterise *how* each architecture fails, not merely *how often*. For this we go beyond point estimates to the shape of per-class performance: the full distribution of per-class F1, the structure of the pairwise confusion matrix, and a class-level dominance comparison. These diagnostics are where the architectural dissociation becomes visible, and the bulk of the subsection below is devoted to them.

5.2. Dataset

TLFS23 (Singaravel et al., 2023) contains 255,155 images across 248 labelled classes (247 Tamil characters plus a Background class), collected at fixed 640×480 resolution under controlled but variable lighting. Per-

channel pixel statistics (mean $[0.507, 0.505, 0.494]$, std $[0.067, 0.063, 0.072]$) confirm near-uniform illumination consistent with the controlled capture protocol. To fit the I/O and storage constraints of the Colab environment used for training, we work with a stratified per-class subset of 200 images per class (49,646 images total). The Background class is filtered out and labels are remapped to contiguous indices in $[0, 246]$, giving a 247-class task. Final splits are 34,612 training, 7,417 validation, and 7,417 test images. The resulting class distribution is near-uniform, with a maximum-to-minimum imbalance ratio of 1.2 and no class below 50% of the mean.

Empirical complexity grouping: Our error-structure analysis benefits from a per-class measure of visual difficulty that is independent of any model. We compute one from the data itself. For each class we sample five images, resize to 32×32 , flatten to a 3072-dimensional pixel vector, and average across samples to obtain a class-mean vector. Pairwise cosine similarity between class-mean vectors gives a 247×247 similarity matrix; each class’s mean similarity to all others summarises how visually confusable it is with its neighbours. The observed distribution is tightly concentrated (mean 0.967, std 0.010), which reflects strong inter-class overlap on this fine-grained task. We split the label space into three tiers at the 33rd and 67th percentiles: *simple* ($n = 82$, mean similarity < 0.965), *moderate* ($n = 84$, $0.965\text{--}0.970$), and *complex* ($n = 81$, ≥ 0.970). 2D PCA and t-SNE projections (Appendix A, Figure 5) show that complex classes cluster more tightly and overlap more than simple classes, which suggests the partition captures real pixel-space structure rather than an arbitrary cut. Two points matter here. The partition is derived from raw pixels, not a learned representation, so it is architecture-agnostic and does not favour either model’s analysis. And it measures visual confusability in pixel space, not a linguistic taxonomy, some Uyir-Mei compounds fall in *simple*, some isolated consonants in *complex*. Pixel-space confusability is the quantity directly relevant to image-based classification difficulty, which is what we want. We use raw-pixel cosine similarity rather than a learned embedding or linguistic taxonomy to keep the difficulty axis architecture-agnostic.

Splits: We use a 70/15/15 train/val/test partition stratified by class label so every class appears in every split. Stratifying by label rather than by complexity tier is deliberate: per-class F1 at test time requires every class to have test instances, which coarser group-level stratification could violate. The resulting splits are nevertheless balanced with respect to complexity tier (approximately 33%/33%/34%). Split files, class indexing, and the complexity map are saved as shared artefacts and loaded unchanged by both architecture pipelines, so the comparison reads from numerically identical train/val/test partitions.

Table 1. Test-set performance on 7,417 held-out images across 247 Tamil characters under matched training regimes.

Metric	EfficientNet-B3	Swin-Small
Top-1 accuracy	0.5177	0.9935
Top-5 accuracy	0.8738	0.9993
Macro-F1	0.4666	0.9935
Weighted-F1	0.4667	0.9935
Top-1/Top-5 gap	0.3561	0.0058
Parameters	11.08M	48.84M
Input resolution	300 × 300	224 × 224
McNemar’s test: $\chi^2(1) = 3497.27, p < 0.001$		

5.3. Evaluation metrics and baselines

We report Top-1 and Top-5 accuracy, Macro-F1 (unweighted average of per-class F1), and Weighted-F1 (support-weighted average). Macro and Weighted F1 agree exactly under perfect class balance and diverge when a model’s errors concentrate on a subset of classes with different support, so the Macro-vs-Weighted comparison is itself diagnostic. For per-class analysis we report the full distribution of per-class F1 across 247 classes, aggregated by complexity tier in box form and compared point-by-point across architectures via a scatter and a difference histogram. For pairwise error analysis we report the top 20 off-diagonal confusion pairs (true $\rightarrow$ predicted, ranked by count) for each model and visualise the full 247×247 confusion matrix.

The dataset paper (Singaravel et al., 2023) itself reports no benchmark models. A small body of prior published Tamil SLR work exists, which we situate against our matched-regime comparison in Section 5.5; none of those works evaluates modern deep architectures on the full 247-class task with per-class error analysis. The principal comparison in this paper is therefore internal: EfficientNet-B3 serves as the CNN baseline against which the Swin Transformer is evaluated under the matched regime of Section 4, isolating architectural effects as cleanly as the mandated pipeline differences (resolution, normalisation, augmentation) allow. To confirm that the observed performance gap is not attributable to random variation, we apply McNemar’s test (McNemar, 1947) to the paired per-image predictions on the held-out test set.

5.4. Overall test performance

Table 1 summarises overall test performance. Swin-Small beats EfficientNet-B3 by 47.58 points on Top-1 and 12.55 points on Top-5. The gap is statistically significant: McNemar’s test on the 7,417 paired predictions yields $\chi^2(1) = 3497.27, p < 0.001$, with 3,544 images correctly classified by Swin but not EfficientNet-B3 against only 15 in the reverse direction, a 236:1 asymmetry that rules out chance as an explanation. A $\approx 4\times$ parameter advantage and a differ-

ence in input resolution cannot, on their own, produce 47 points of Top-1 separation on a single fine-tuning task. The gap reflects a difference in the kind of representation each architecture builds, not only in the capacity with which it builds it, and the rest of this section supports that reading.

Two patterns inside Table 1 are themselves informative. EfficientNet-B3’s Top-5 accuracy (0.8738) is far higher than its Top-1 (0.5177): when the CNN gets the top prediction wrong, the correct class is usually within the next four. That is what you would expect from a model that recognises coarse hand-shape category but cannot discriminate within it. For Swin, both metrics sit near ceiling, so the gap is uninformative. The second pattern is that EfficientNet-B3’s failures appear distributionally uniform across the label space, evidenced primarily by within-tier per-class F1 dispersion of $\sigma \approx 0.21$ – 0.26 in every tier (Table 3; full distributions in Appendix E, Figure 15), and confirmed by the agreement between Macro-F1 and Weighted-F1 to four decimals. We note that under near-perfect class balance (imbalance ratio 1.2) the latter agreement is constrained regardless of error structure, so dispersion is the stronger evidence. Swin’s two values also agree (both 0.9935), but at the opposite end of the scale: it succeeds uniformly.

5.5. Comparison with prior published work

Table 2. Comparison with published Tamil SLR systems. “This work” rows report Top-1 accuracy on the TLFS23 held-out test set (7,417 images, 247 classes).

Work	Method	Classes	Acc.
Rajam & Bal. (2012)	Finger encoding	31	98.1%
Tamil SLR (2024)	CNN	247	85.0%
Prabhu et al. (2024)	MediaPipe + RF	247	96.5%
Sangeetha & DG (2025) [†]	CNN + Bi-LSTM	13	95.0%
This work (EffNet-B3)	Fine-tuning	247	51.8%
This work (Swin-Small)	Fine-tuning	247	99.4%

We situate our results against four published Tamil SLR works (Table 2). Two structural observations matter more than the headline numbers.

Class-set differences dominate the comparison. Three works evaluate on subsets that exclude the 216 Uyir-Mei compounds: Rajam and Balakrishnan (2012) use a 31-class vowel/consonant set with binary finger-position encoding; Sangeetha and Divya Gowri (2025), despite being the only prior work to use TLFS23 itself, restrict their evaluation to 13 Uyir Ezhuthukal vowel classes; and Tamil SLR (2024) does not disclose its class count or compositional coverage. The compositional confusion topology that drives our error-structure analysis cannot be exposed on a 13 or 31-class subset – the classes that produce it are not present.

Table 3. Mean per-class F1 by empirical complexity tier. Tiers are defined in Section 5.2 from inter-class cosine similarity in pixel space.

Tier	n	EffNet F1	EffNet σ	Swin F1	Swin σ
Simple	82	0.4849	0.2635	0.9943	0.0110
Moderate	84	0.4459	0.2449	0.9935	0.0101
Complex	81	0.4732	0.2059	0.9927	0.0118

No prior work reports per-class or pair-level error structure. Prabhu et al. (2024) report 96.5% on a 247-class MediaPipe+Random Forest pipeline, the closest in scale to our setup, but discard the within-hand visual information our error analysis specifically diagnoses (pose keypoints collapse a hand into 21 landmarks, removing exactly the texture and stroke information that distinguishes a base consonant from its modifier). Sangeetha and Divya Gowri (2025) report Top-1 / precision / recall / F1 only and do not analyse per-class dispersion, dominance, or confusion-pair topology. The second contribution of the present paper - making error structure a first-class diagnostic on this label space, is therefore complementary to these accuracy-only baselines.

5.6. Per-complexity-tier means

Table 3 gives per-tier mean F1 with within-tier standard deviation. Both architectures are basically flat across tiers. EfficientNet-B3’s mean F1 sits in a narrow band (0.45–0.49); Swin’s variation is on the order of 10^{-3} . Pixel-space complexity does not strongly predict either architecture’s mean per-tier accuracy. The tier-level summary is the wrong place to look for the dissociation: the aggregate pairwise-similarity metric averages over all 246 other classes per class, washing out the single-pair signal that drives the CNN’s actual failures (its same-base-consonant nearest sibling in each consonant family). Confusion-pair topology, examined in §5.7, exposes this directly.

The revealing number in Table 3 is the within-tier standard deviation. EfficientNet-B3’s per-class F1 has $\sigma \approx 0.21$ –0.26 within each tier, while Swin’s is $\sigma \approx 0.01$ these are an order of magnitude apart. The CNN’s mediocrity is structured, with some classes well-handled and others nearly unrecognised; the Transformer’s success is distributionally tight. We unpack this next.

5.7. Error structure: dispersion, topology, and dominance

Three diagnostics: per-class F1 dispersion, per-class dominance, and confusion-pair topology together characterise a dissociation that the headline numbers and tier means only hint at. The full per-class F1 distributions (boxplots in Appendix E, Figure 15) show that EfficientNet-B3’s per-class F1 spans the entire $[0, 1]$ interval, with quartiles at roughly

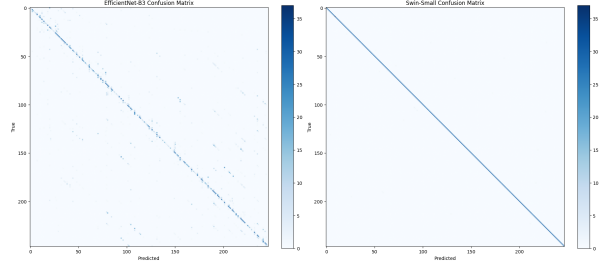


Figure 1. Test-set confusion matrices (247×247) on identical held-out images. EfficientNet-B3 (left) shows structured off-diagonal bands from systematic Uyir-Mei confusions; Swin-Small (right) is essentially diagonal with no cell exceeding count 2.

$[0.20, 0.69]$ and a non-trivial tail of classes at $F1 = 0$. Swin’s per-class F1 is concentrated in the top 2% of the unit interval with extremely tight interquartile ranges across all three tiers. The two architectures are not differently-located versions of the same distribution; their per-class F1 distributions are differently shaped.

Per-class F1 differences (Swin minus EfficientNet) on identical test images and class indices are even more telling. Swin is better by more than 0.01 F1 on 245 of 247 classes; EfficientNet is better by more than 0.01 on 0 of 247; the remaining 2 classes are tied. The mean per-class advantage is 0.527. This is dominance in the technical sense: pointwise superiority across essentially the entire label space rather than a stochastic ordering. No class exists for which the CNN’s locality bias is empirically advantageous so the architectural advantage is uniform in direction, even where it varies in magnitude.

Figure 1 makes the architectural dissociation immediately visible: EfficientNet-B3 shows structured off-diagonal bands absent from Swin-Small.

The third and most informative diagnostic is the topology of EfficientNet-B3’s top errors. Its twenty most-confused class pairs (Appendix E, Table 4) cluster on numerically adjacent class indices, as shown in Figure 1 and Appendix E, Table 4. The TLFS23 label index is constructed by sorting class names with the consonant base as the primary key, so numerically adjacent indices correspond to characters that share a base consonant and differ only in the vowel modifier. In script terms this is the compositional structure of the Uyir-Mei system. EfficientNet-B3’s top errors are not random: they are systematic confusions among compositionally related characters that share a base stroke. Several triangular structures appear within the confusion list: (109, 108), (110, 108), and (228, 108) all collapse onto class 108, which indicates that the CNN treats a whole compositional family as indistinguishable rather than just confusing pairs. A similar pattern appears around 241, with both 242 and 239 mapping to it.

Of those twenty CNN confusion pairs, Swin completely

resolves 19. It makes zero or near-zero errors on those exact ($i \rightarrow j$) directions. The one remaining pair has a small but nonzero confusion count. Swin’s own twenty most-confused pairs are sparse, with maximum count 2 across the test set, below the level at which one can meaningfully speak of a systematic confusion. Broken down by complexity tier, CNN’s top-20 confusions are 9 in moderate, 8 in simple, 2 in complex (and 1 unknown), with 5/20 within the same tier. Compositional confusion structure is therefore largely orthogonal to pixel-space complexity, which is another reason why the tier-mean view in §5.6 masks the architectural difference.

These three diagnostics support one conclusion. EfficientNet-B3 has learned coarse hand-shape categories but cannot discriminate within their compositional sub-structure; Swin has learned the compositional structure itself. These are not differently-tuned versions of the same representation but different kinds of representation.

5.8. Mechanistic diagnosis via Grad-CAM

To diagnose the CNN’s failure mode mechanistically we run Grad-CAM (Selvaraju et al., 2017) on the final convolutional block of EfficientNet-B3, restricted to test images where the CNN predicted incorrectly but Swin predicted correctly such that the failure is attributable to the model rather than to image-level confusability. Six examples (two per complexity tier) are shown in Appendix F, Figure 19.

The pattern is consistent and, once you see it, hard to unsee. On many misclassifications the CNN’s class-discriminative attention drifts off the hand and onto task-irrelevant background: the back of the chair the signer sits in, the signer’s torso and clothing, and most memorably a printed shirt logo that attracts the peak activation outright. In several examples the peak activation falls outside the hand region entirely. None of the training labels are conditioned on clothing or furniture. Nothing in the data justifies attending to those features. The CNN reaches for them anyway. Of the 18 misclassified examples examined across the Grad-CAM figures in Appendix F, peak activation falls outside the hand region entirely in at least 10 cases, with a further 5 showing activation concentrated on clothing or chair rather than the hand.

This fits the locality story. When within-hand information is itself highly discriminative, local features suffice and EfficientNet attends where it should. When the discriminating signal is a small modifier shared across compositional Uyir-Mei pairs, the same locality bias pulls attention toward whatever else in the frame is locally distinctive, signal-relevant or not. Background drift is a property of an inductive bias that permits a shortcut this task does not support.

5.9. Discussion

A 47.58-point Top-1 gap between two pretrained backbones fine-tuned on the same 34,612-image training set requires explanation. Three candidate accounts exist and are not mutually exclusive.

Capacity: Swin-Small has $4.4\times$ as many parameters as EfficientNet-B3 (48.84M vs. 11.08M). Capacity arguments predict the *level* of accuracy a model can reach but not the *shape* of its failure: per-class F1 dispersion ($\sigma \approx 0.24$ vs $\sigma \approx 0.01$), near-equality of Macro and Weighted F1, and a confusion-pair topology that clusters on numerically adjacent same-base-consonant indices are qualitative properties of *where* the model fails, not just how often. Capacity scales monotonically; topology does not. The EfficientNet scaling experiments of Tan and Le (2019) themselves report sub-linear accuracy gains across B0 through B7 (a roughly $30\times$ parameter range) on ImageNet, so a $4\times$ parameter advantage at the B3/Swin-Small scale would be expected to contribute several accuracy points rather than tens, and would not be expected to alter qualitative error structure. Capacity is therefore one contributor to the level gap but cannot account for the dissociation in shape that this section foregrounds.

Residual training-regime differences: The pipelines differ in augmentation strategy and label smoothing. These could plausibly contribute a few accuracy points but cannot account for the gap: augmentation differences shift the *location* of a per-class F1 distribution, not its shape, and the distributions we observe are shaped very differently. A separate concern is the EfficientNet-B3 training schedule itself. Stage 1 ran for 10 epochs with the head still improving when the budget expired (validation Top-1 reached 0.097, with no plateau), and Stage 2 began from this undertrained head at 10^{-5} , which is conservative for a head that had not yet converged. A longer Stage 1 budget or a higher Stage 2 head learning rate would likely raise EfficientNet-B3’s absolute F1 by several points. We do not, however, expect such a shift to alter the structural finding, because the diagnostics that define it: per-class F1 dispersion, dominance, and the compositional confusion topology clustering on same-base-consonant indices reflect the locality prior rather than the training budget. This is a falsifiable prediction: a CNN trained to convergence on full-data TLFS23 should show a higher absolute F1 but the same compositional confusion topology. If it does not, capacity and training were the real drivers and the architectural claim weakens.

Architectural inductive bias: The error-structure diagnostics provide evidence that the first two accounts alone cannot explain. EfficientNet-B3’s failures concentrate on compositionally related class pairs the structural property that distinguishes TLFS23 from datasets on which CNN performance approaches ceiling. Swin resolves 19 of the CNN’s 20 most systematic confusions. Those pairs are not

harder in any general sense; they are compositionally hard, and a model that integrates stroke and modifier simultaneously does not confuse them. The statistical confirmation via McNemar’s test ($\chi^2(1) = 3497.27, p < 0.001$) rules out the possibility that this pattern reflects sampling variation rather than a genuine architectural difference. The practical implication for Indic SLR has two parts. First, headline Top-1 accuracy on a compositionally structured label space is misleading in isolation: a model can report $\sim 50\%$ Top-1 while failing systematically on the linguistically most important (Uyir-Mei) classes. Second, error-structure diagnostics expose this failure where coarse statistics hide it. We recommend their adoption as routine reporting practice for Indic SLR benchmarking, and expect the same qualitative pattern broad CNN dispersion with compositional confusion topology against tight transformer dispersion to replicate on other Brahmi-derived scripts.

Our claim is narrow: under matched fine-tuning on TLFS23, the architectures produce qualitatively different error structures interpretable in terms of their inductive biases, supported by Grad-CAM evidence of where the CNN’s locality prior fails. We do not claim CNNs are intrinsically incapable of compositional learning.

What would falsify this: Three concrete predictions follow. First, an EfficientNet-B3 trained to convergence on the full $\approx 1,000$ -image-per-class TLFS23 should show higher absolute F1 but the same compositional confusion topology if full-data training dissolves the topology rather than merely raising the level, the locality-prior explanation weakens. Second, a parameter-matched comparison (EfficientNet-B7 vs Swin-Small, or EfficientNet-B3 vs Swin-Tiny) should preserve the dispersion and topology dissociation while shrinking the level gap; if matching capacity also dissolves the structural difference, capacity rather than inductive bias is the real driver. Third, the same dissociation should appear on other compositional Indic scripts; if it does not, the finding may reflect TLFS23-specific dataset artefacts rather than a general property of locality priors on compositional label spaces.

5.10. Limitations

Generalisation: TLFS23 was collected under controlled conditions from a bounded signer pool against plain backgrounds. The Swin accuracy we report is therefore an in-distribution upper bound, not a deployable performance estimate; out-of-distribution evaluation across signers, lighting, and viewpoints is required before any deployment claim, consistent with broader SLR findings (Rastgoo et al., 2021).

Data scale and training schedule: We train on 200 images per class rather than the full $\approx 1,000$, and the EfficientNet-B3 schedule itself is conservative (Stage 1 head still improving at the 10-epoch cap; Stage 2 backbone LR

10^{-5}). Both would lift the CNN’s absolute F1 but, by the argument in Section 5.9, are not expected to alter the structural findings.

Splitting protocol: The 70/15/15 split is stratified by class label rather than by signer, so frames from the same signer performing the same gesture may appear in both training and validation. The close Swin val/test agreement (99.73% vs 99.29%) bounds val/test inflation but does not rule out train→val leakage; a signer-disjoint evaluation is the appropriate next step.

Interpretability and scope: Grad-CAM transfers cleanly to convolutional features but not to shifted-window self-attention, so the CNN diagnostic is rigorous while the transformer side lacks a matched figure (attention rollout would close this asymmetry). And we evaluate static fingerspelling rather than continuous signing whether the dissociation survives temporal context is open.

6. Conclusion

We presented, to our knowledge, the first published deep-learning benchmark on the full 247-class TLFS23 task, comparing EfficientNet-B3 and Swin-Small under matched fine-tuning regimes. The 47.58-point Top-1 gap is statistically significant (McNemar’s test, $\chi^2(1) = 3497.27, p < 0.001$). Beyond this gap, the architectures dissociate in error structure: EfficientNet-B3’s per-class F1 is broad and uniformly mediocre ($\sigma \approx 0.24$), while Swin’s is tight and uniformly near-perfect ($\sigma \approx 0.01$). Swin dominates on 245 of 247 classes and resolves 19 of EfficientNet-B3’s 20 most systematic confusion pairs that in TLFS23’s class ordering share a base consonant. Grad-CAM on EfficientNet-B3 failures shows attention drifting onto task-irrelevant background, giving a mechanistic account of where the CNN’s locality prior fails. For compositionally structured Indic SLR label spaces, error topology and per-class F1 dispersion are more informative than headline Top-1 accuracy, and we recommend their adoption in future benchmarking. The architectural account is falsifiable: it predicts that a parameter-matched, fully-trained CNN should preserve the compositional confusion topology, and that the same dissociation should replicate on other compositional Indic scripts.

6.1. Future work

While the current study focuses on static gesture recognition, true sign language translation is inherently fluid. A natural extension of this work is to adapt the architecture to interpret temporal sequences. To account for this new temporal axis, the model structure could be updated to include Long Short-Term Memory (LSTM) units or Gated Recurrent Units (GRUs). These recurrent layers, specifically through their forget gate mechanisms, would allow the

model to learn the transitions between finger shapes, bridging the gap between static image classification and real-time video translation for Tamil Sign Language.

References

- Abnar, S. and Zuidema, W. Quantifying attention flow in transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Hounsby, N. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Duarte, A., Palaskar, S., Ventura, L., Ghadiyaram, D., DeHaan, K., Metze, F., Torres, J., and Giro-i-Nieto, X. How2Sign: A large-scale multimodal dataset for continuous American sign language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- Isaacs, J. and Foo, S. Hand pose estimation for American sign language recognition. In *Thirty-Sixth Southeastern Symposium on System Theory*, pp. 132–136. IEEE, 2004.
- Jayasinghe, L. et al. Automated vision-based recognition of Sri Lankan Tamil sign language fingerspelling. In *IEEE International Conference on Industrial and Information Systems*, 2011.
- Karthikeyan et al. Recognition and translation of Tamil sign language using image processing. 2022.
- Khan et al. A systematic review of sign language recognition approaches: From classical methods to deep learning. *Multimedia Tools and Applications*, 2024.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin Transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022, 2021.
- Mangamuri, L. S. T., Jain, L., and Sharma, A. Two hand Indian sign language dataset for benchmarking classification models of machine learning. 2019.
- Patel et al. Advancements in sign language recognition: A survey of methodologies and future directions. 2024.
- Prabhu, S. et al. Tamil alphabet sign language recognition using hand landmark detection. 2024.
- Rajam, P. S. and Balakrishnan, G. Recognition of Tamil sign language alphabet using image processing to aid deaf-dumb people. *Procedia Engineering*, 30:861–868, 2012.
- Rajkumar, R. et al. Tamil sign language recognition using machine learning techniques. *IRJET*, 2019.
- Rastgoo, R., Kiani, K., and Escalera, S. Sign language recognition: A deep survey. *Expert Systems with Applications*, 164:113794, 2021.
- Sangeetha, M. and Divya Gowri, C. Deep learning-based Tamil sign language assistance for speech disabilities. In *2025 International Conference on Next Generation Computing Systems (ICNGCS)*. IEEE, 2025.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.
- Sharma et al. Deep learning based Indian sign language recognition for human–robot interaction. 2024.
- Singaravel et al. TLFS23: A large-scale dataset for Tamil fingerspelling sign language recognition. *Data in Brief*, 2023.
- Singh et al. Review of Indian sign language recognition systems and accessibility applications. *Asian Journal of Electrical Sciences*, 2023.
- Machine learning-based Tamil sign language recognition tool for the deaf and hard-of-hearing community. In *IEEE Conference Publication*, 2024. IEEE Xplore document 10452538.
- Tan, M. and Le, Q. V. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6105–6114, 2019.
- McNemar, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Statement of Individual Contributions

This project was a collaborative effort, with each team member contributing to distinct but complementary components of the research pipeline:

- **63046** — Built the EDA and preprocessing pipeline, including the empirically grounded complexity grouping via inter-class cosine similarity, complexity-aware tiered augmentation, and the stratified 70/15/15 split shared across both architectures. Fine-tuned

EfficientNet-B3 end-to-end across 247 classes, covering staged backbone unfreezing and hyperparameter selection. For the cross-architecture comparison, implemented per-class and per-complexity-group F1 diagnostics and the zoomed confusion-matrix analysis quantifying how many of the CNN’s top-20 confused pairs Swin resolved. Contributed to report writing and compiled the appendix.

- **64552** — Conducted the literature review and identified the research gaps motivating the benchmark. Designed and implemented the Swin Transformer pipeline, including the two-stage transfer-learning protocol, the hybrid TensorFlow/PyTorch training setup, the cosine annealing schedule, and the classification head configuration. Built the initial cross-architecture comparison framework, unified test pipeline, aligned prediction arrays, and per-class F1 diagnostics and ran McNemar’s test for statistical significance of the architectural performance gap. Contributed to report sections on transformer architecture, model specification choices, and the compilation of results and interpretation.
- **61904** — Contributed to the EfficientNet-B3 pipeline, including Stage 1 and Stage 2 training, training-curve plotting, test-set evaluation, per-complexity-group F1 scores, confusion matrices, and Grad-CAM analyses. Wrote the CNN notebook markdown documentation and the corresponding sections of the report. Proofread and edited the final report.
- **60133** — Contributed to the comparative evaluation of the CNN and Transformer architectures, including analysis of training dynamics, per-complexity-tier performance, and overall test-set results. Led the comparative Grad-CAM analysis aimed at identifying root causes of CNN misclassifications. Researched and added future work section. Assisted with testing the Transformer pipeline. Contributed to the results and discussion sections of the report and proofread.

During implementation, AI was consulted occasionally for debugging specific code errors, with all fixes understood and verified by the responsible team member before integration. AI was not used to write, structure, or generate any substantive portion of the codebase.

For the written report, AI was used solely for minor grammatical checks. All analysis, interpretation, and framing is the team’s own work.

AI did not conceive the research question, design the pre-processing pipeline, choose evaluation metrics, define the complexity grouping framework, or interpret any experimental results.

7.1. Links to AI Chats Used

<https://chatgpt.com/share/69f24ab5-ba58-83eb-aa4f-dd997bdfe960>
<https://chatgpt.com/share/69f25040-0994-83eb-8aa8-92b45b7a0545>
<https://chatgpt.com/share/69f2504d-9e00-83eb-af47-6002f14295a4>
<https://chatgpt.com/share/69f25070-d47c-83eb-8740-5f750af744d7>
<https://chatgpt.com/share/69f26c1d-b4b4-83eb-b6e1-400360159310>
<https://chatgpt.com/share/69f26c60-5410-83eb-a321-458a7277e6f3>
<https://chatgpt.com/share/69f270a8-f8e4-83eb-acf6-0c4c4ca8de52>
<https://chatgpt.com/share/69f2729d-45a8-83eb-8c74-e013bc9f1aa5>
<https://chatgpt.com/share/69f274ba-97fc-83eb-a1a8-79b4ef6c19e4>
<https://chatgpt.com/share/69f27936-6780-83eb-a658-593474f708f1>

7. Acknowledgement of AI Usage

AI assistance was used in a limited and clearly delineated capacity across this project. Early project ideation, including exploring potential research directions and identifying the Tamil sign language recognition gap, was developed through discussion with ChatGPT (OpenAI). The core research framing, specifically the CNN vs Transformer comparative analysis across character complexity groups, was conceived independently and brought to the conversation. AI served as a sounding board during this stage; all final decisions on research framing, dataset selection, and methodology were made independently by the team.

Appendix

A. Exploratory Data Analysis

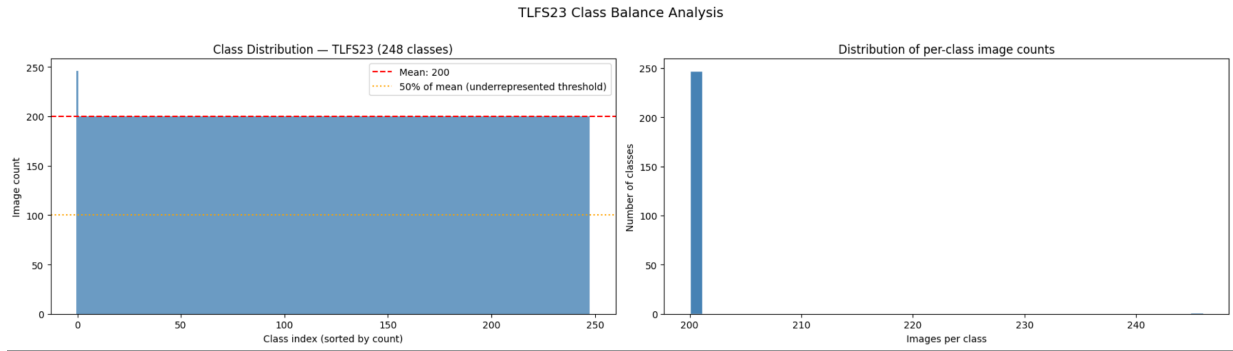


Figure 2. TLFS23 class balance after subset sampling at 200 images per class. Maximum-to-minimum imbalance ratio 1.2; no class below 50% of the mean.

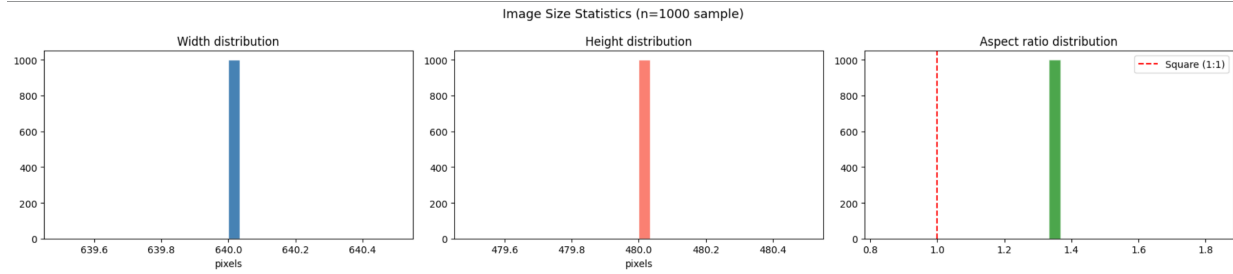


Figure 3. Image-size statistics on a 1,000-image sample from TLFS23. All images are fixed 640×480 with constant aspect ratio (≈ 1.33), which confirms the controlled-capture protocol described by Singaravel et al. (2023).

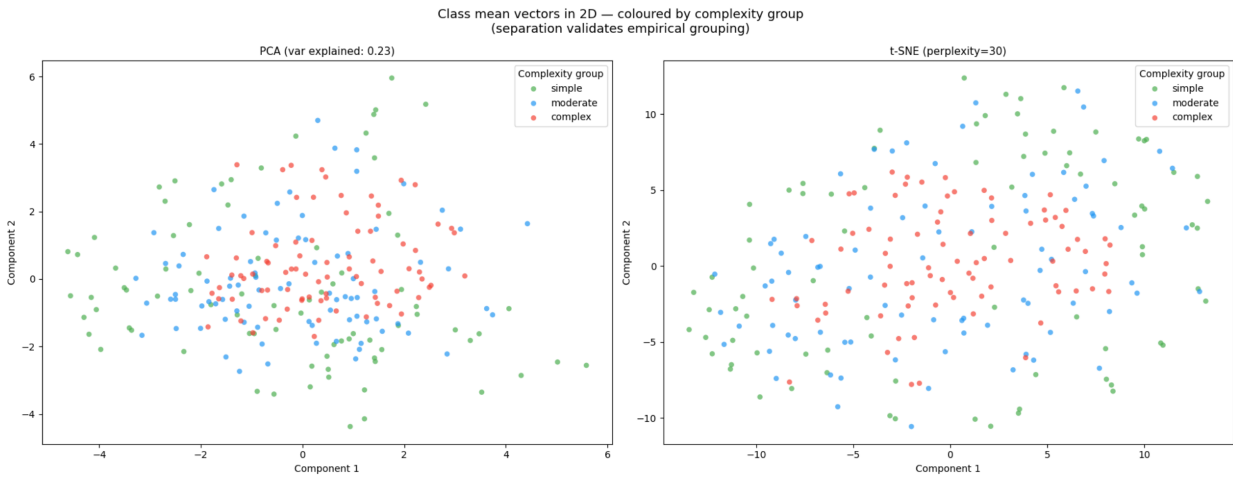


Figure 4. Inter-class visual similarity analysis. Mean similarity 0.967, standard deviation 0.010. The vertical lines on the histogram mark the 33rd-percentile (simple/moderate boundary, 0.965) and 67th-percentile (moderate/complex boundary, 0.970) thresholds used to define the empirical complexity tiers.

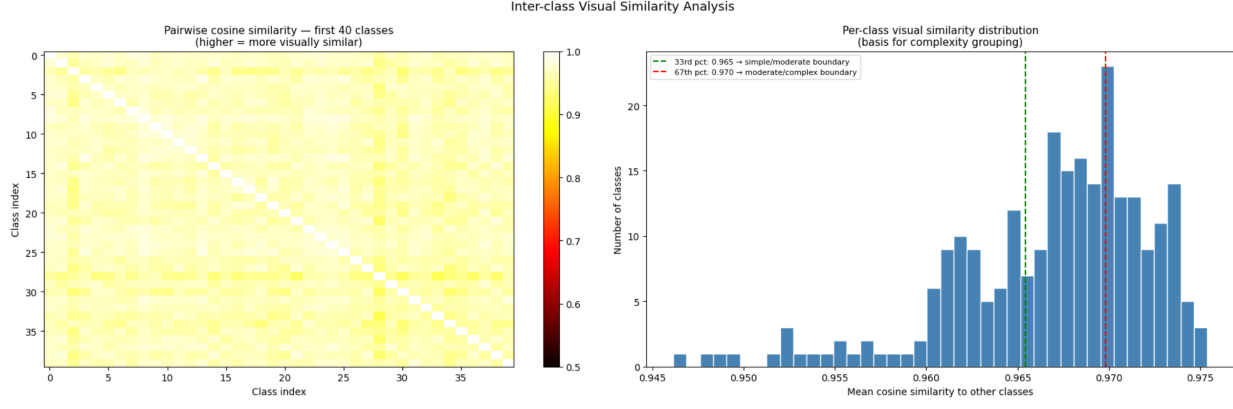


Figure 5. Low-dimensional projections of per-class mean pixel vectors, coloured by complexity tier. PCA variance explained at two components: 23%. Complex classes cluster more densely and overlap more than simple classes, which is consistent with the empirical partition capturing real pixel-space structure.

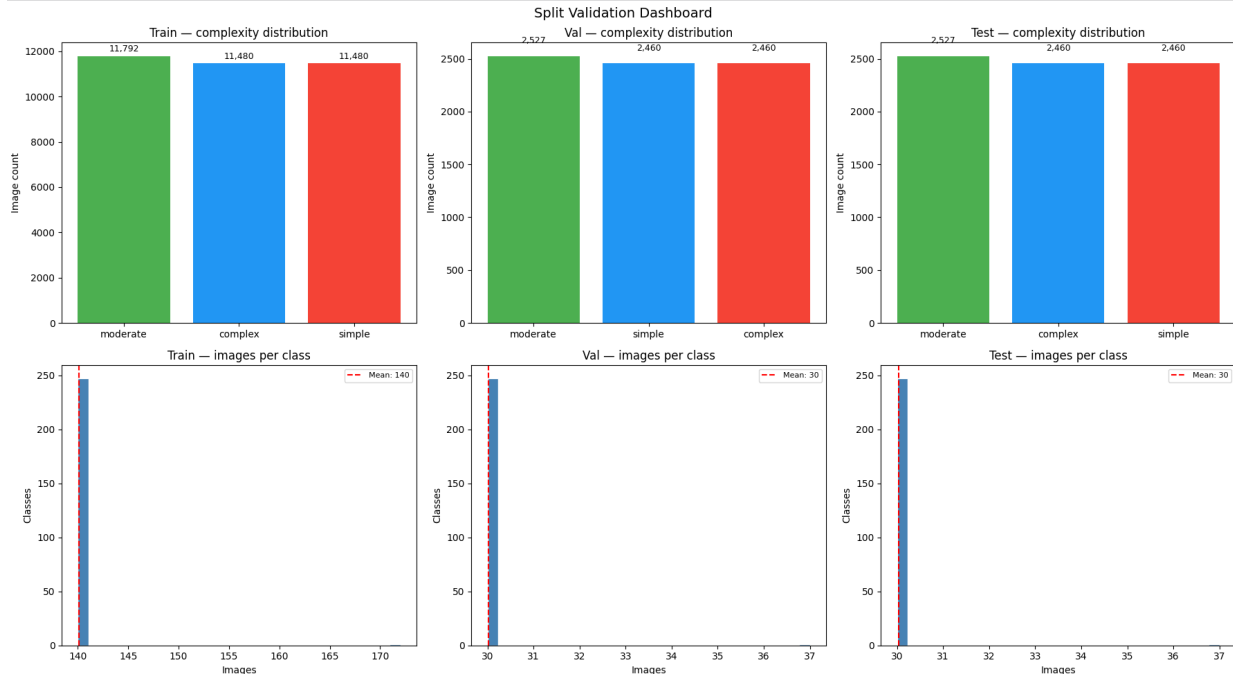


Figure 6. Split validation. Top row: image counts by complexity tier in train/val/test, with balanced complexity distribution across all three splits. Bottom row: per-class image-count histograms for each split, confirming that 70/15/15 stratification by class label produces uniform per-class support (mean 140 train, 30 val, 30 test) with negligible spread.

B. Augmentation Validation

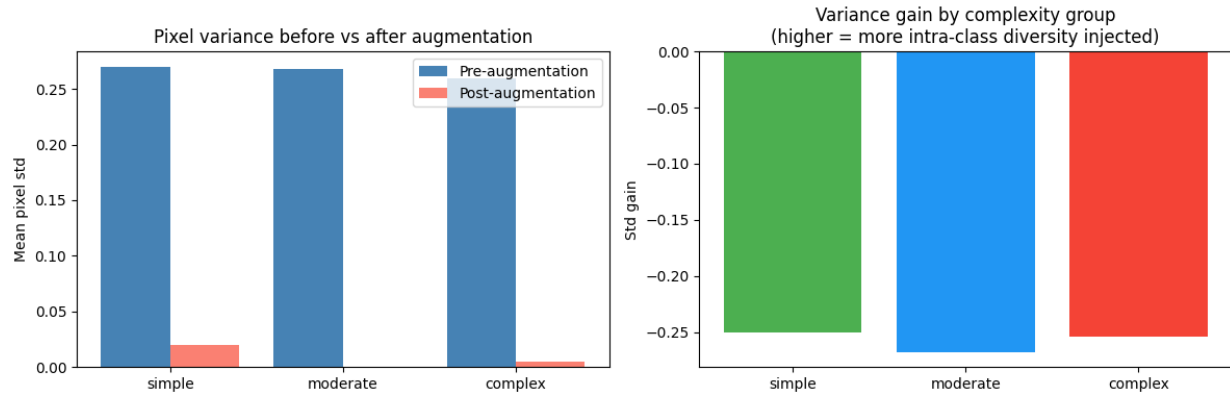


Figure 7. Pre- versus post-augmentation pixel statistics by complexity tier. Augmentation transforms are applied at training time; the figure compares intra-class variance before and after augmentation on a sample batch.

C. EfficientNet-B3 Training

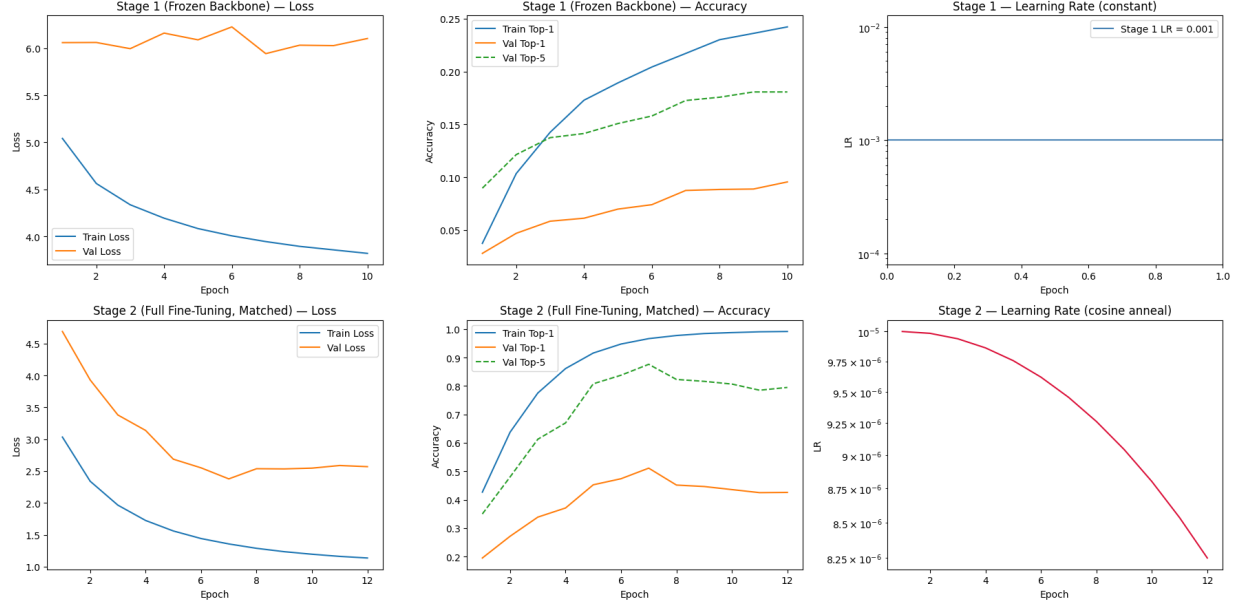


Figure 8. EfficientNet-B3 training dynamics across both stages. Top row: Stage 1 (frozen backbone) loss, Top-1/Top-5 accuracy, and constant learning rate 10^{-3} . Bottom row: Stage 2 (full fine-tuning) loss, Top-1/Top-5 accuracy, and cosine-annealed learning rate. Stage 2 hits the early-stopping criterion within 12 epochs.

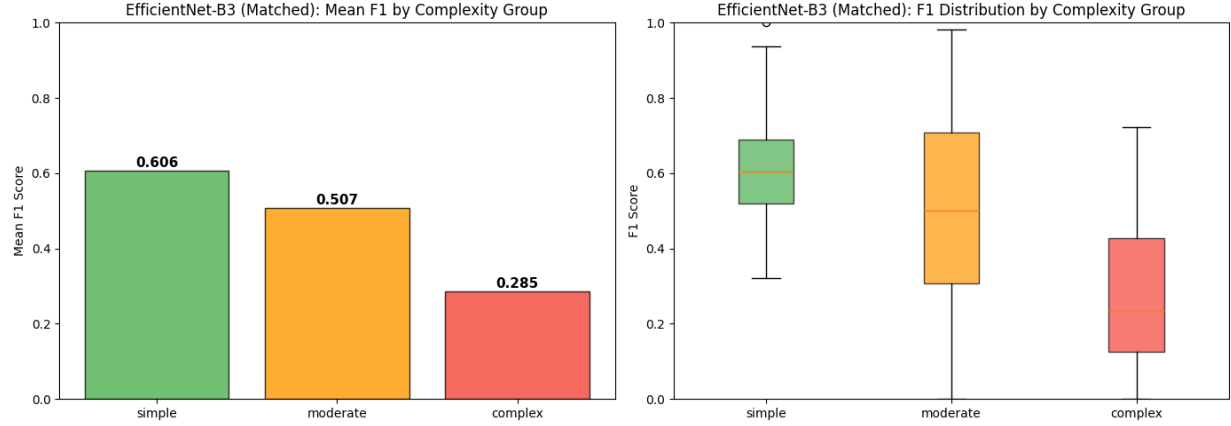


Figure 9. EfficientNet-B3 per-complexity-tier mean F1 (left) and per-class F1 distribution (right). Mean F1 declines from simple (0.606) to moderate (0.507) to complex (0.285), and within-tier dispersion is large in every tier. (The simple/moderate/complex aggregation here uses a finer-grained internal split than the matched 82/84/81 partition used for the cross-architecture comparison; the qualitative dispersion pattern is the same under either partition.)

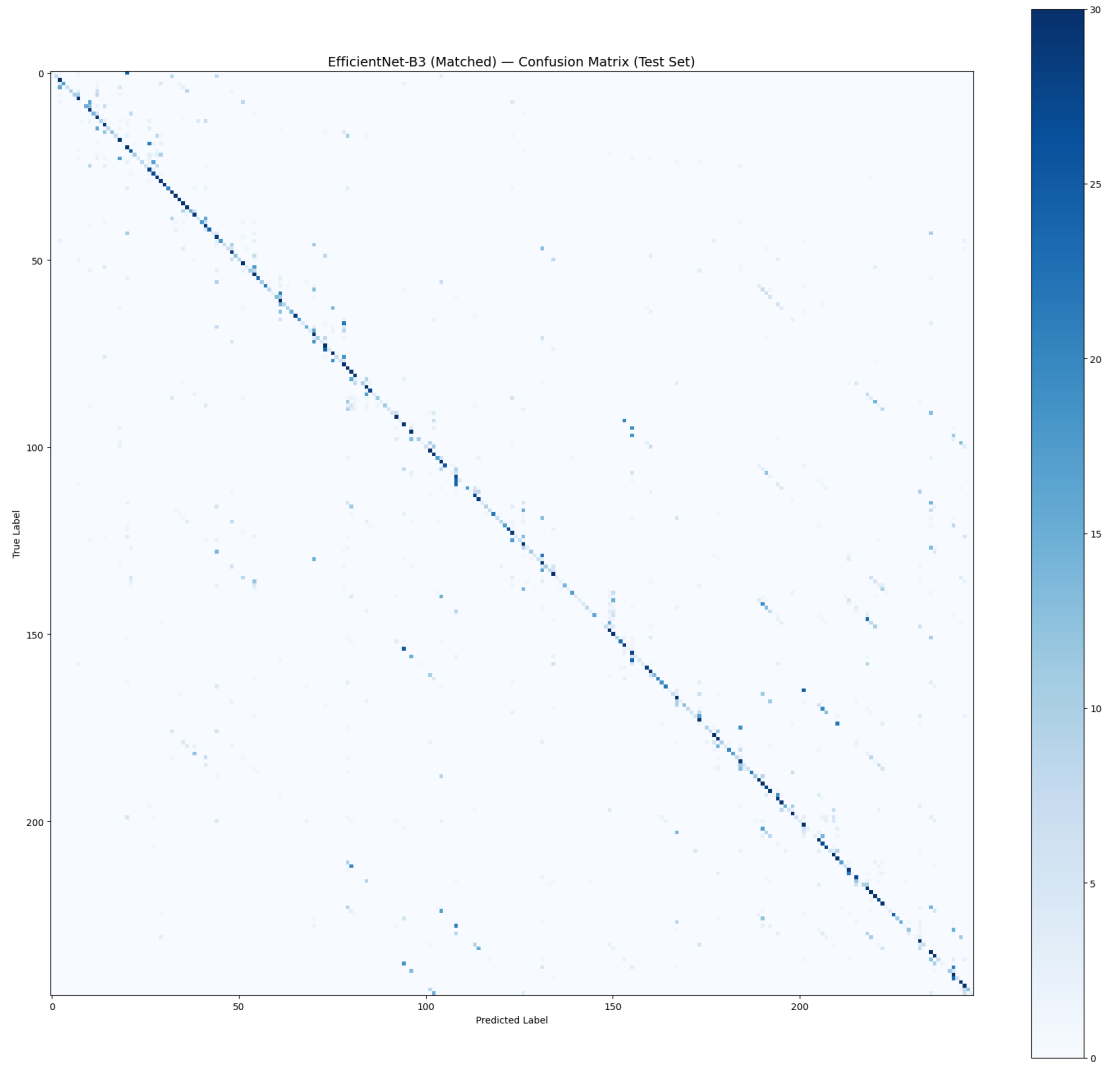


Figure 10. EfficientNet-B3 test-set confusion matrix (247×247). Diagonal structure is visible, but off-diagonal clusters correspond to the systematic confusions among compositionally related classes analysed in §5.7.

D. Swin Transformer Training

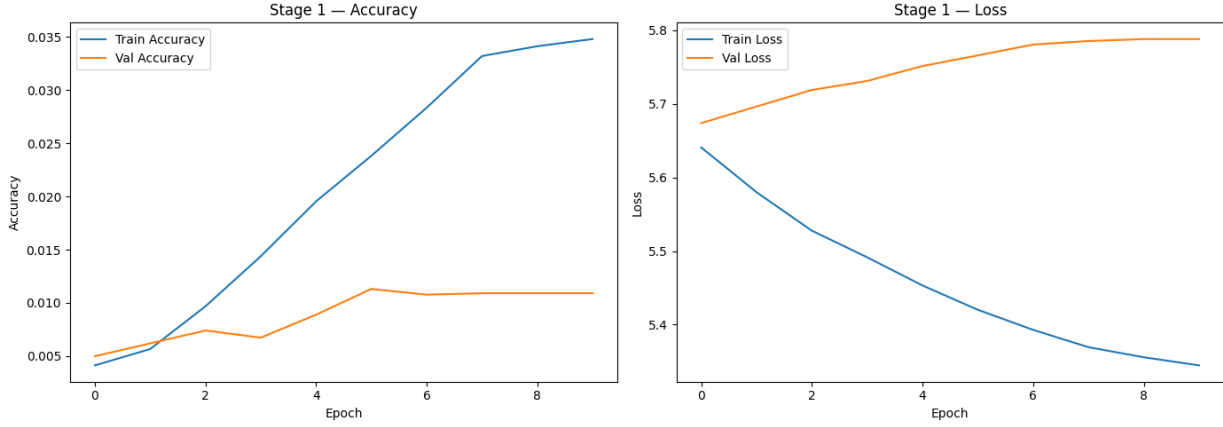


Figure 11. Swin-Small Stage 1 training (cached features, head only). Train accuracy reaches $\approx 3.5\%$ and validation accuracy plateaus near 1% within 10 epochs, which shows that the frozen Swin features are not directly linearly separable on TLFS23 as expected for ImageNet features on a 247-class fine-grained Tamil-character task and that Stage 2 unfreezing is essential.

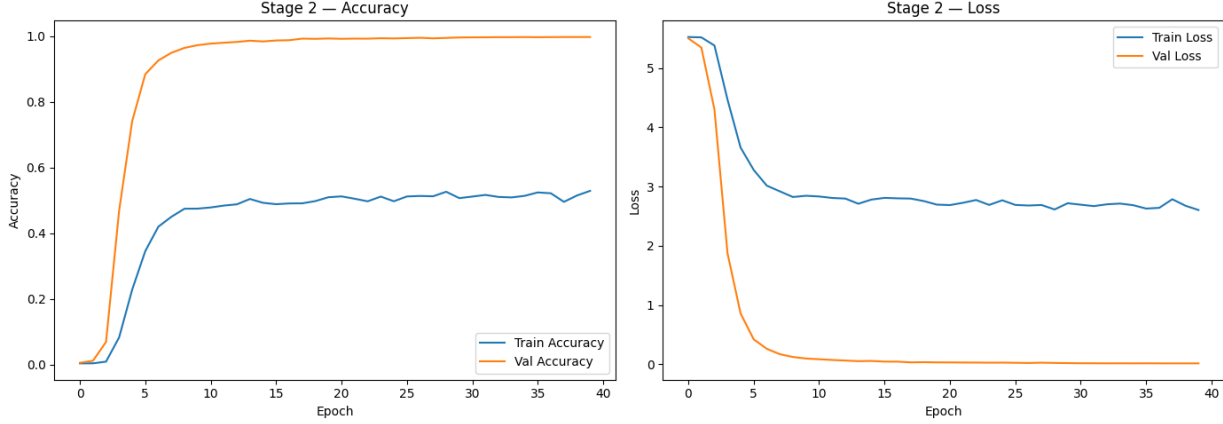


Figure 12. Swin-Small Stage 2 training (full end-to-end fine-tuning). Validation accuracy reaches $\approx 99.7\%$ by epoch 15 while training accuracy plateaus near 50%. The training-validation gap reflects on-the-fly augmentation making training images numerically harder than clean validation; the close val/test agreement (99.73% vs 99.29%) confirms the result is not an overfit to the validation set.

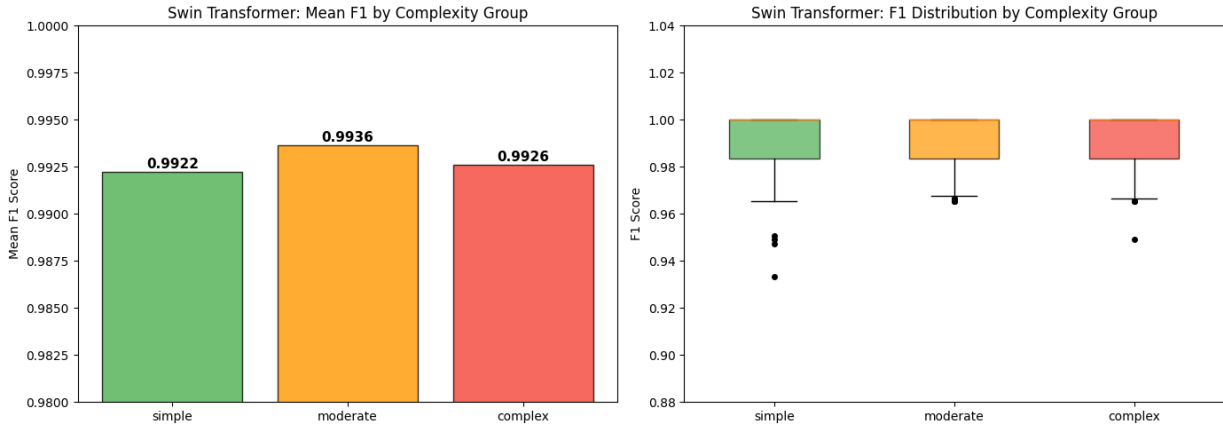


Figure 13. Swin-Small per-complexity-tier mean F1 (left) and per-class F1 distribution (right). Mean F1 is essentially flat across tiers (0.9922–0.9936), and the per-class distribution is concentrated in $[0.95, 1.00]$ in every tier distributionally tight, in direct contrast to the EfficientNet-B3 distributions in Figure 9.

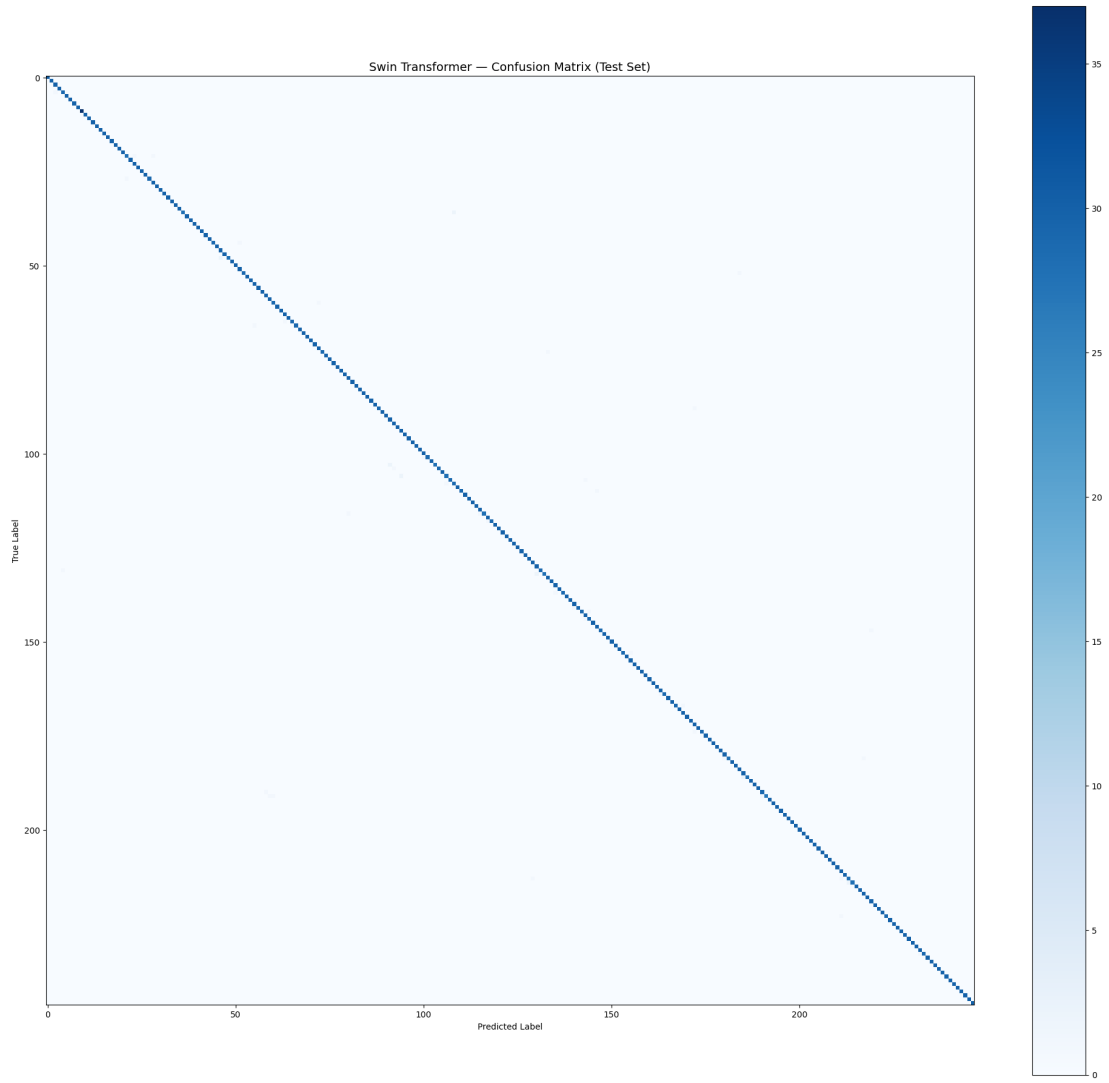


Figure 14. Swin-Small test-set 247×247 confusion matrix. The matrix is essentially diagonal; no off-diagonal cell exceeds count 2.

E. Per-Class and Per-Pair Comparison

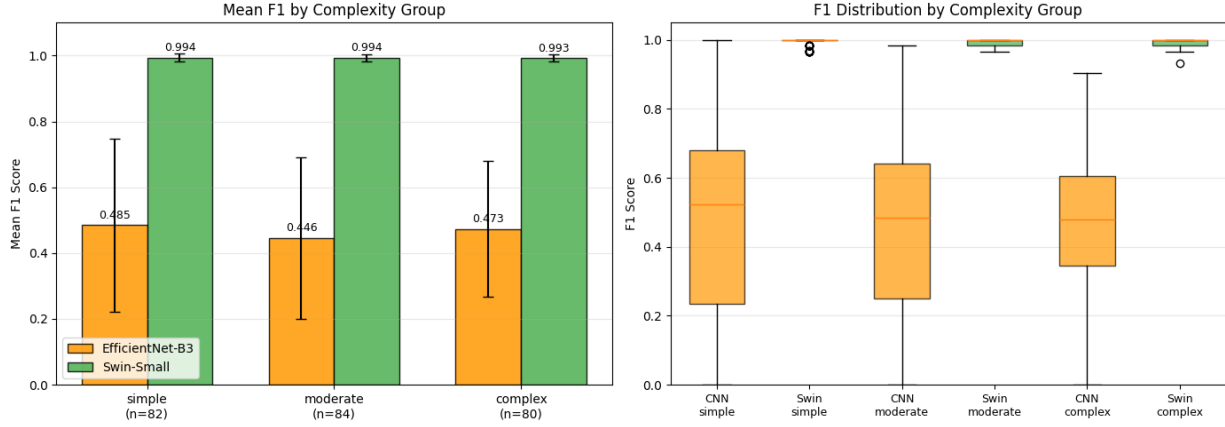


Figure 15. Per-tier mean F1 (left) and per-class F1 distribution (right) for both architectures on the matched 82/84/81 complexity partition. The dissociation is in distribution shape, not tier means: EfficientNet boxes span $\approx [0.20, 0.69]$ with whiskers reaching zero; Swin boxes sit in $[0.98, 1.00]$.

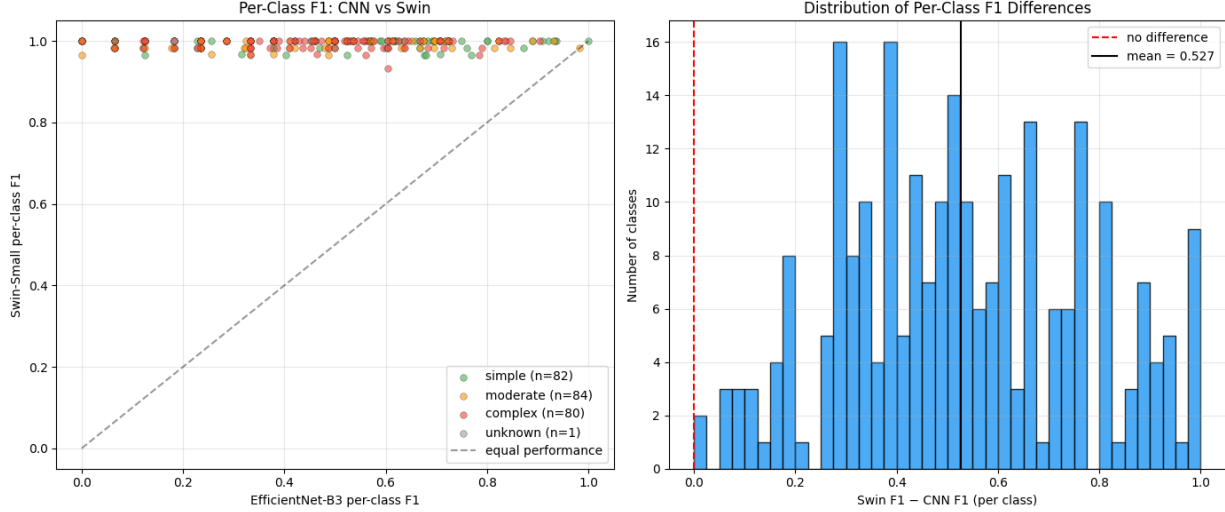


Figure 16. Per-class F1 dominance. Left: scatter of per-class F1 (CNN x -axis, Swin y -axis), coloured by complexity tier, with $y = x$ reference line. Every point sits above the line. Right: histogram of per-class F1 differences (Swin - CNN). Swin beats EfficientNet-B3 on 245 of 247 classes (gap > 0.01 F1), loses on 0 of 247, and ties on 2. Mean per-class F1 advantage 0.527.

Table 4. Top 20 most-confused class pairs for EfficientNet-B3, with corresponding Swin per-class F1 on the true class. Swin completely resolves 19 of these 20 systematic CNN confusions.

True	Pred	Count	CNN F1 (true)	Swin F1 (true)
0	20	25	0.182	1.000
165	201	25	0.125	1.000
154	94	25	0.125	1.000
109	108	24	0.235	1.000
74	73	24	0.235	1.000
157	155	24	0.286	1.000
110	108	24	0.000	1.000
146	218	23	0.000	1.000
242	241	23	0.333	1.000
67	78	22	0.235	1.000
129	131	22	0.229	0.984
174	210	22	0.065	0.983
19	26	21	0.065	1.000
59	61	21	0.125	0.984
214	213	21	0.125	0.966
228	108	21	0.062	0.984
239	241	21	0.000	1.000
170	206	20	0.378	1.000
212	80	20	0.324	1.000
93	153	20	0.182	1.000

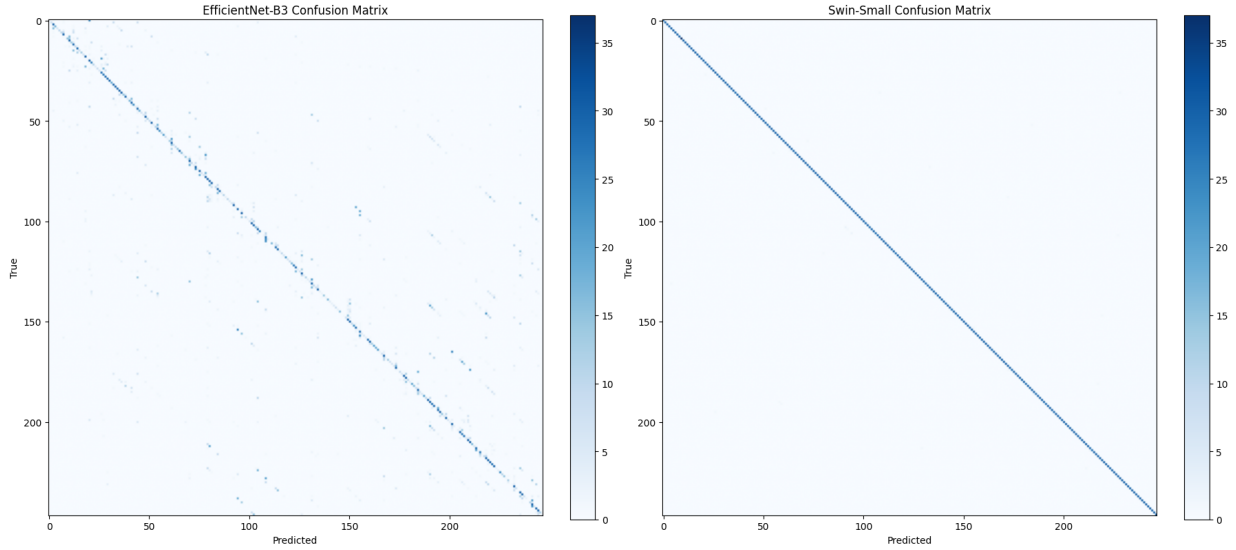


Figure 17. Side-by-side full 247×247 confusion matrices on identical test images. EfficientNet-B3 (left) shows structured off-diagonal mass secondary bands and isolated bright cells far from the diagonal that is absent in Swin-Small (right).

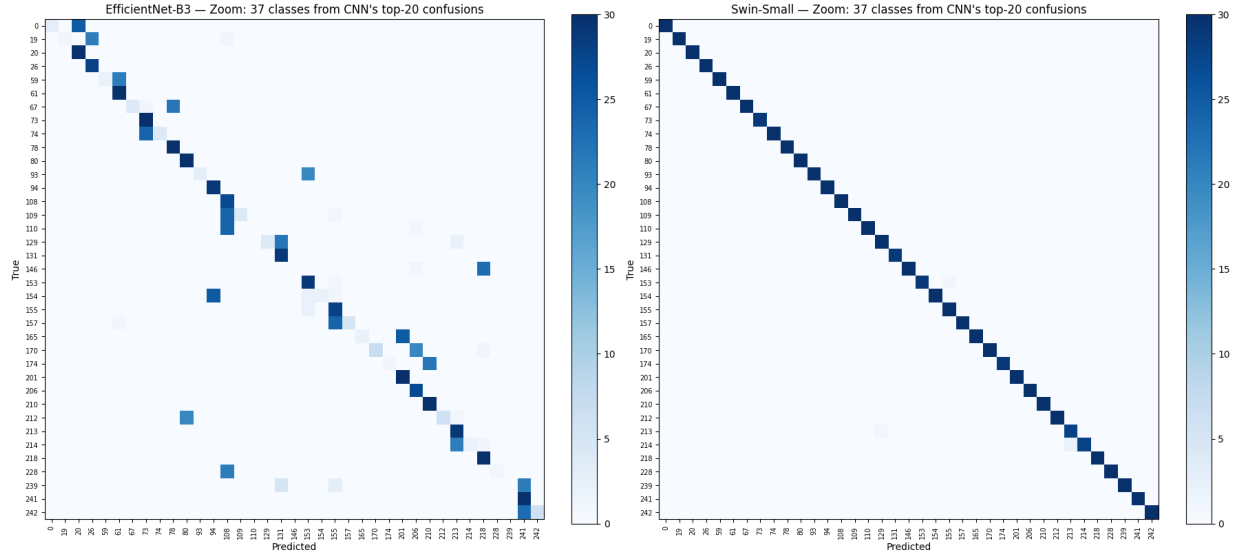


Figure 18. Zoom view: confusion matrices restricted to the 37 classes involved in EfficientNet-B3’s top-20 confusion pairs. Each off-diagonal block in the EfficientNet panel corresponds to one of the confusions in Table 4; the Swin panel is essentially diagonal on the same 37 classes, which is the “Swin resolves 19/20” result visualised.

F. Grad-CAM Visualisations

All figures in this section show Grad-CAM overlays restricted to test images where the model under analysis predicted incorrectly. Each panel pairs the original image with its class-discriminative activation map; warmer colours (red/yellow) indicate regions of higher gradient-weighted activation.



Figure 19. EfficientNet-B3 Grad-CAM on cases where the CNN failed but Swin succeeded. Each row shows two examples from one complexity tier (top: simple, middle: moderate, bottom: complex), with the original image paired with its Grad-CAM overlay. Attention drift is plain: the top-left example has peak activation on the signer’s Qatar Airways shirt logo rather than on either hand; the top-right example also places peak activation on printed shirt text; both moderate-tier examples show activation bleeding off the hand onto chair edges and lower torso; several examples have no peak activation inside the hand region at all. The CNN’s locality bias reaches for whatever local texture is most distinctive in the frame shirt prints, chair edges, clothing folds because the within-hand signal separating the true and predicted classes is too subtle for local receptive fields to isolate.

Grad-CAM: Misclassified Examples by Complexity Group
Each row shows 2 samples with Original + Grad-CAM overlay

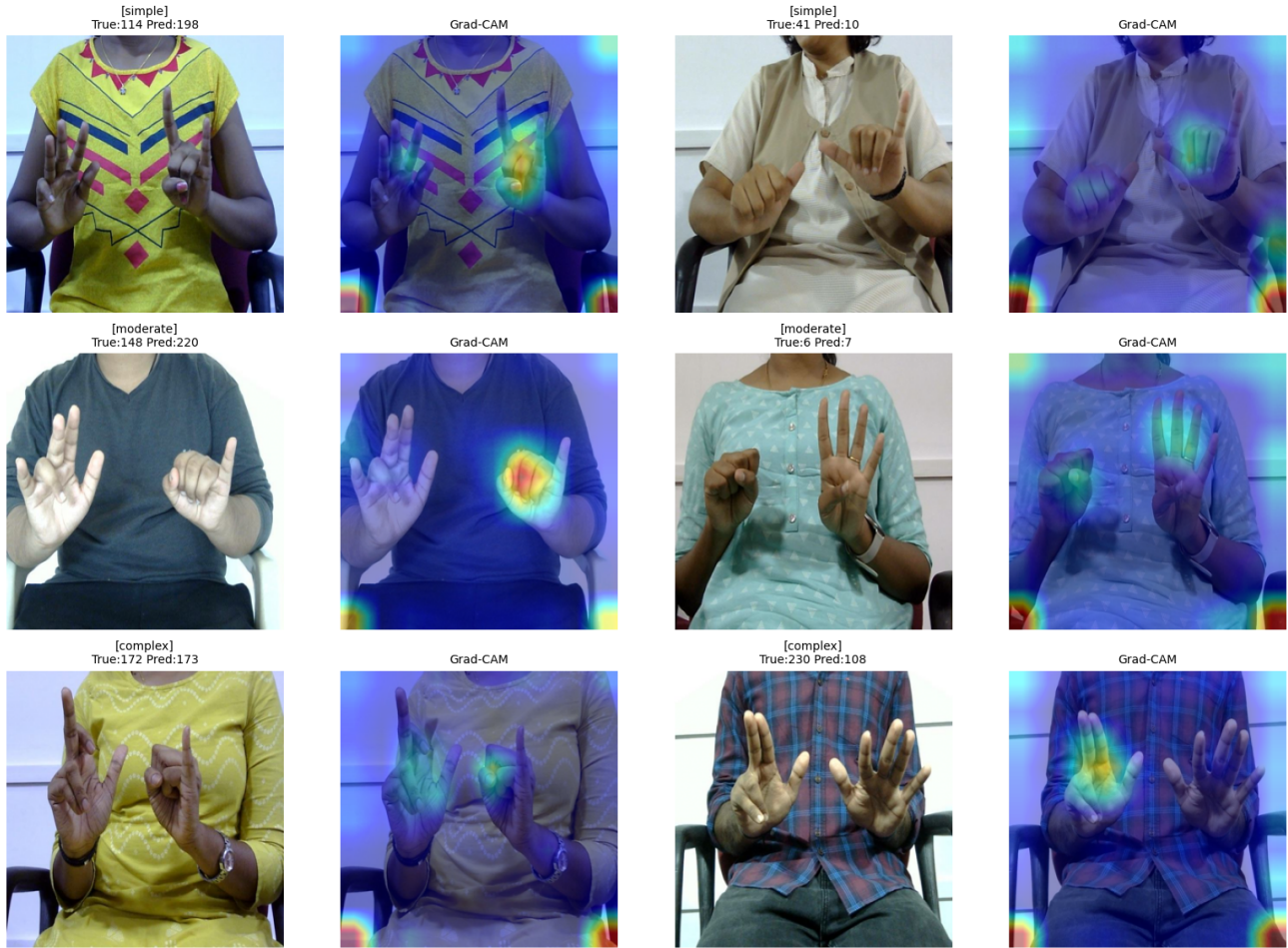


Figure 20. EfficientNet-B3 Grad-CAM on a second set of misclassified examples, organised by complexity tier (simple: top row, moderate: middle row, complex: bottom row). Each group shows the original image alongside its Grad-CAM overlay. Across all tiers the activation pattern is consistent with the locality-drift failure mode: peak activations land on clothing texture, torso, and background rather than on the discriminating hand region, confirming that the convolutional locality prior cannot resolve the subtle modifier-level differences that distinguish compositionally related Tamil characters.

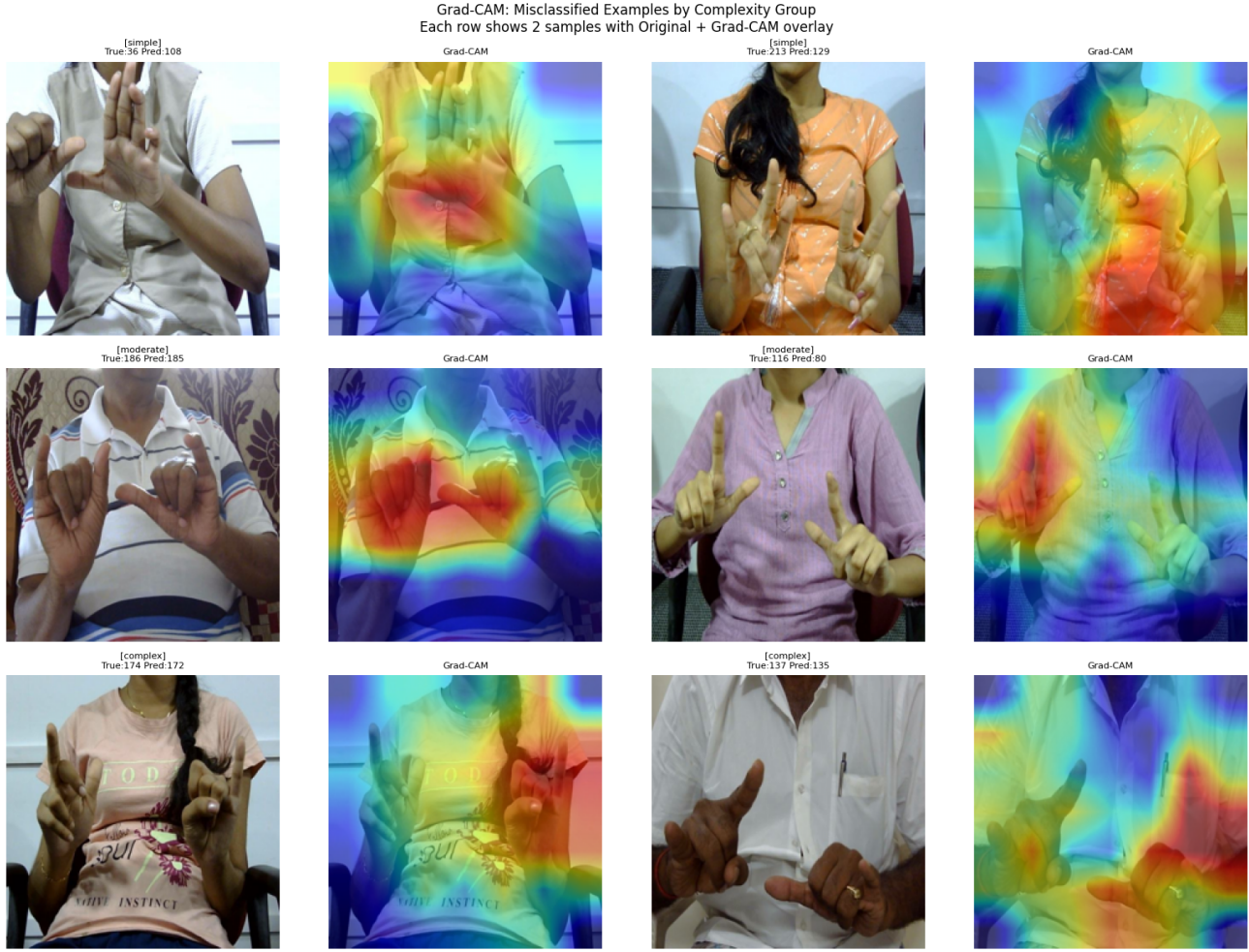


Figure 21. Grad-CAM applied to the Swin Transformer on misclassified examples (same layout as Figures 19 and 20: simple top, moderate middle, complex bottom). Although Grad-CAM is not natively suited to shifted-window self-attention, the maps are shown here for qualitative comparison only. In contrast to EfficientNet-B3, activations are more diffuse and less concentrated on task-irrelevant background regions, broadly consistent with the Transformer’s ability to integrate global context rather than relying on locally distinctive texture.